



PROJECT REPORT

Rabbit Hole

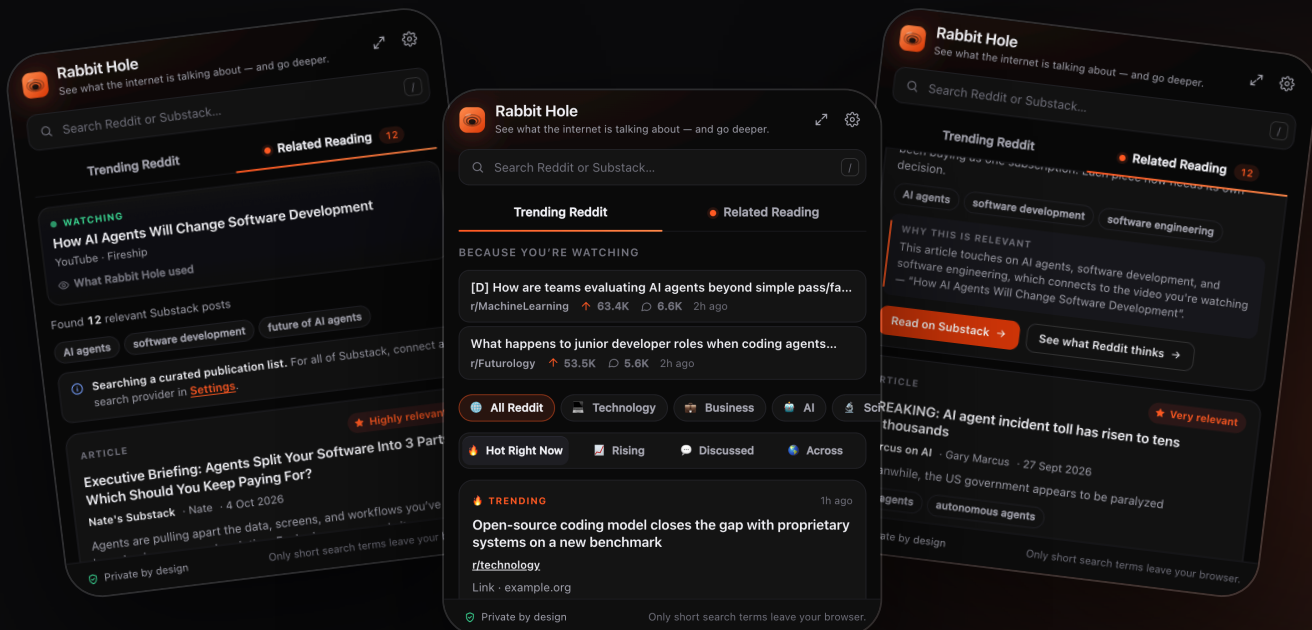
See what the internet is talking about — and go deeper.

Chrome · Manifest V3

TypeScript · React 19

Reddit × Substack

Privacy-first



Contents

	Executive summary	3
CH 1	Problem, goals and users	4
CH 2	Product tour	6
CH 3	UX and visual design	10
CH 4	System design	12
CH 5	Flowcharts	15
CH 6	Privacy and security	22
CH 7	Testing and quality assurance	23
CH 8	Usability evaluation plan	25
CH 9	Limitations, risks and future work	27
APP	Appendices	28

Companion deliverables (in `docs/`): `Rabbit-Hole-Heuristic-Evaluation-Form.pdf` — fillable usability & heuristic evaluation pack · `flowcharts/` — project flowchart, system architecture, Figma-ready UI flow (SVG/PNG/Mermaid) · `screens/` — every UI state as PNG.

Executive summary

Rabbit Hole is a Chrome extension (Manifest V3) that turns *what you are watching* into *what to read next*. It pairs two views of the internet — what is moving on **Reddit** right now, and thoughtful long-form writing on **Substack** related to a YouTube video or Netflix title — and bridges them: any thread can lead to deeper reading, and any article can lead to the discussion around it.

230

automated tests
in 17 files, all passing

28/28

real-Chrome end-to-end
checks (Chrome 155)

61

health-checked Substack
publications (keyless
mode)

715 KB

production build;
content scripts 6–9 KB
each

What was built

- **Trending Reddit** — threads ranked by *momentum* (upvote and comment velocity, recency, cross-community spread and measured growth), not total votes; four sections, ten topic filters, instant cache-first rendering.
- **Related Reading** — a context pipeline detects the video or title, derives topics and entities, generates focused queries and ranks Substack posts with a transparent 0–100 relevance score, always with a plain-language “*Why this is relevant*”.
- **Privacy as a feature** — nothing is read until the extension is opened, only on four sites, with per-platform switches, a “What Rabbit Hole used” disclosure, RAM-only caches and a minimal permission set.
- **Resilience** — Reddit blocks anonymous traffic and Substack has no public search API, so each has a provider chain with graceful degradation and honest messaging instead of fabricated data.

How it was verified

Unit and integration tests cover ranking, extraction, caching and failure handling; a release gate proves development keys cannot reach a production bundle; and an end-to-end suite loads the *built* extension in a real Chrome and drives the real popup against the real service worker. A usability-evaluation pack (heuristic evaluation, cognitive walkthrough, think-aloud test, SUS, accessibility audit, error-state matrix and A/B plan) accompanies this report as a fillable form.

Scope note

Live Reddit data could not be verified from the build machine (Reddit answered 403 to JSON and 429 to feeds); that path is verified against a captured real feed and mocked responses. Netflix detection is verified against fixtures that mirror its markup, and the optional AI layer against a stubbed API transport. See §9.1.

Problem, goals and users

1.1 Problem statement

Curiosity strikes while watching: a video mentions a technology, a series raises an idea. Following it means leaving the page, guessing a search query, and sorting through results that are optimised for clicks. Meanwhile, social discussion (Reddit) and considered writing (Substack) live in separate places, and Reddit's front pages reward *total votes* rather than what is *gaining momentum*. There is no lightweight bridge between “what everyone is discussing now” and “what thoughtful people have written about this”.

1.2 Goals and non-goals

#	Goal	Measure of success
G1	Surface what is trending on Reddit by momentum, with growth indicators	Fresh fast-growing threads outrank older high-total threads (tested)
G2	Understand what the user is watching on YouTube / Netflix without breaking terms or playback	Extraction on watch pages, Shorts, episodes; graceful “can’t identify” fallback
G3	Recommend relevant Substack writing and explain every pick	Relevance 0–100 with labels; grounded “Why this is relevant”
G4	Bridge Reddit $\rightleftharpoons$ Substack	“Find deeper reading →” and “See what Reddit thinks →” on every card
G5	Privacy first: no history, no tracking, minimal permissions, user control	Permission audit, per-platform switches, RAM-only caches, release gate
G6	Keep working when data sources are blocked, limited or offline	Provider chain + circuit breaker; stale-if-error; friendly error states
G7	Fast, accessible, polished	Instant skeleton paint; keyboard operable; WCAG-minded contrast; reduced motion

Non-goals: scraping Reddit or Substack HTML; bypassing Netflix DRM or authentication; collecting browsing history; replacing a search engine; mobile browsers.

1.3 Target users (illustrative personas)

The curious viewer

Aanya, 20 · CS undergraduate

Watches technical YouTube and series. Wants the “why” behind what she sees, in readable essays, without a research detour.

Needs: instant relevance, explanations, low effort

The trend watcher

Rahul, 34 · product manager

Scans Reddit for what is gaining traction in his field. Distrusts raw vote counts; wants to know what is *rising*.

Needs: momentum signals, topic filters, speed

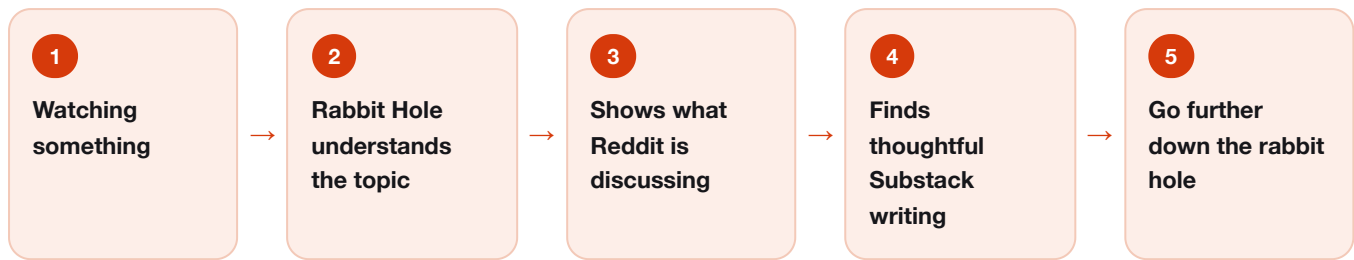
The privacy-conscious reader

Meera, 41 · journalist

Reluctant to install extensions that “read every page”. Will adopt only if she can see and control exactly what is read and sent.

Needs: transparency, control, minimal permissions

1.4 The core journey



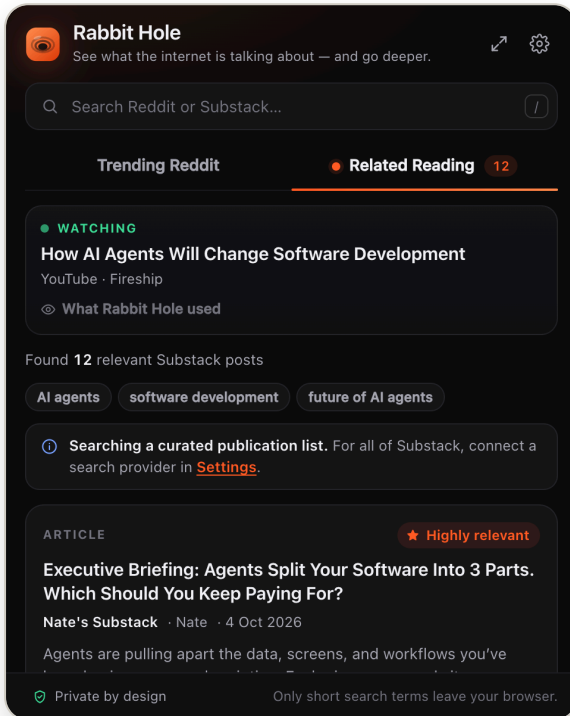
1.5 Requirements

ID	Requirement	Status	Evidence
FR1	Trending Reddit with four sections and topic filters	Done	redditRanking tests; e2e
FR2	Detect YouTube video / Short and Netflix title / episode	Done	extractor tests; e2e on fixture pages
FR3	Manual search and “What are you watching?” fallback	Done	contextManager tests; e2e
FR4	Substack discovery: search API + curated RSS fallback	Done	failureHandling, parsers tests
FR5	Relevance 0–100 with labels and explanations	Done	relevance tests (18)
FR6	Reddit ⇌ Substack bridges	Done	UI flow screens 12, 13, 21, 22
FR7	Settings: Reddit, Related Reading, Privacy, Appearance, Integrations, Data	Done	e2e (toggle persists)
FR8	Optional AI layer (topics, queries, judging)	Done (opt-in)	anthropicProvider tests (stubbed)
NFR1	Cache aggressively; de-duplicate; cancel stale requests	Done	cache tests (17)
NFR2	Strict CSP; validated, sanitised external data	Done	manifest + release gate + e2e
NFR3	Minimal permissions	Done	manifest tests; e2e
NFR4	Keyboard and screen-reader support	Done — audit pending	Usability form, Method F

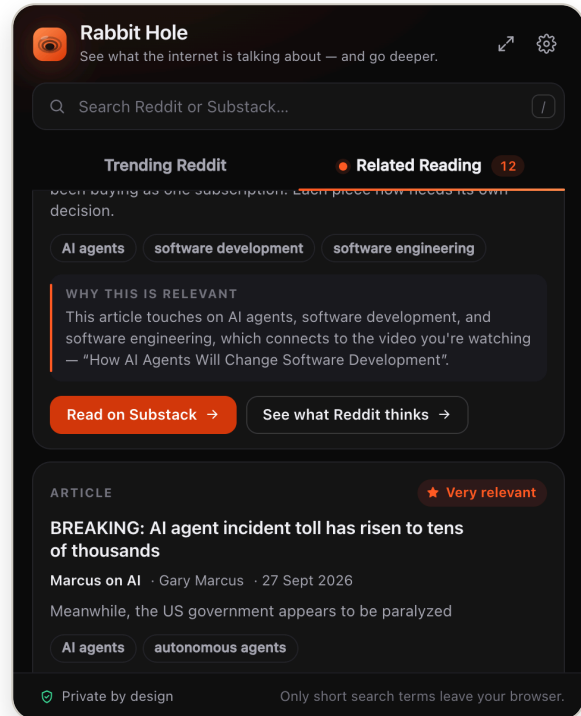
Product tour

Every screen below is a capture of the built interface (480 × 600 px, Chrome's maximum popup height). Reddit threads are **illustrative sample data**; Substack articles come from real public RSS feeds.

2.1 Related Reading — the core experience

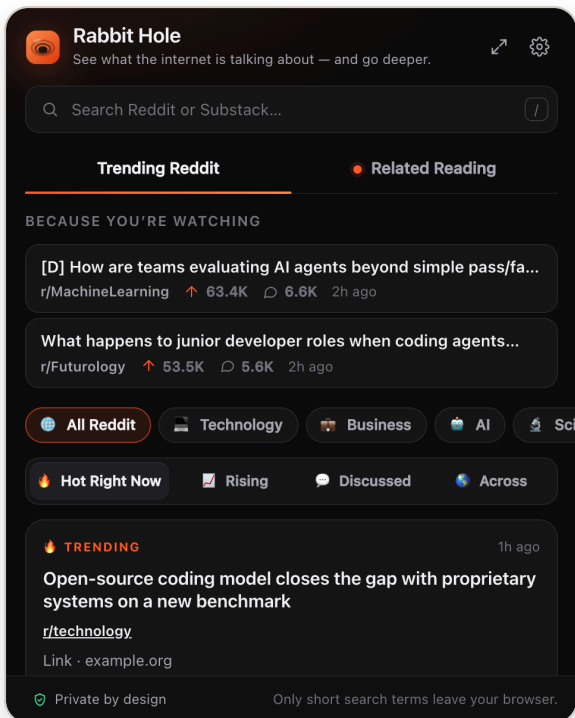


Context bar says what was detected (“Watching — YouTube · Fireship”). Topics searched are shown; a banner is honest that only a curated list is searched without a search key.

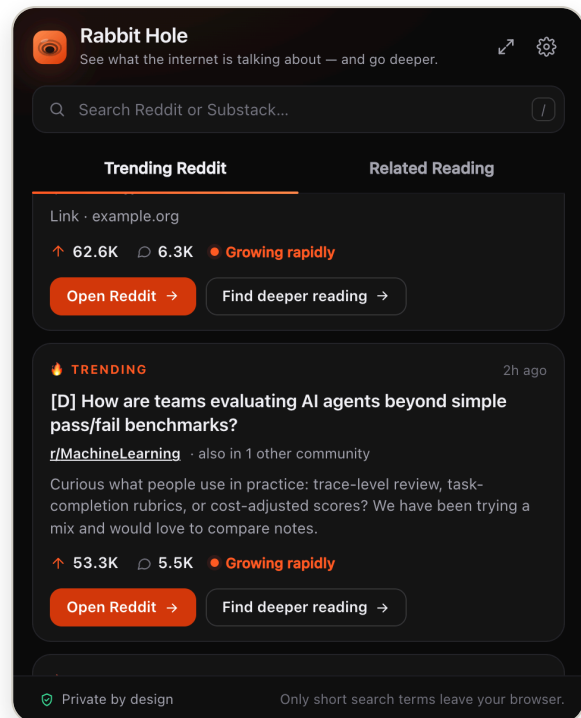


Article card: relevance label, publication · author · date, excerpt, matched topics and a grounded *Why this is relevant*. Two actions: read, or see what Reddit thinks.

2.2 Trending Reddit — momentum, not just votes

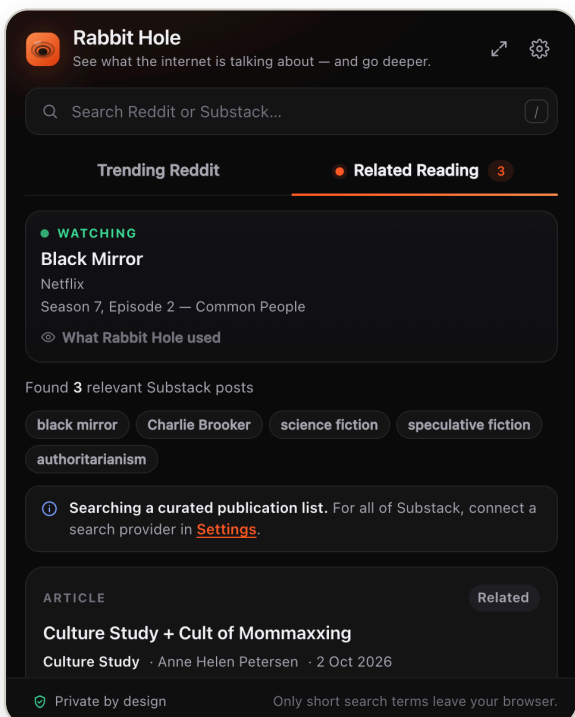


Because you're watching shows threads related to the video as compact rows. Topic chips filter by community group; four sections rank by trending score.

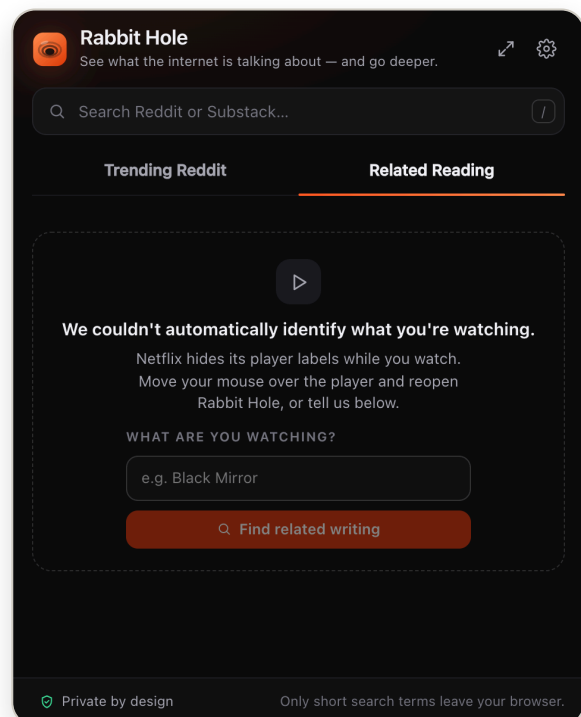


Thread cards: subreddit, preview, ↑ upvotes, comments and a *growth label* (e.g. “Growing rapidly”) derived from measured velocity.

2.3 Netflix and the honest fallback

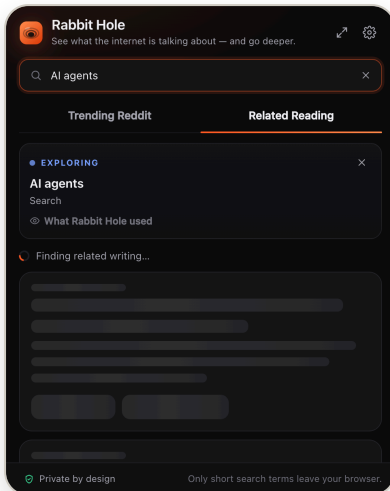


Episode detected from the player label (Black Mirror · Season 7, Episode 2); themes expand to genre concepts.

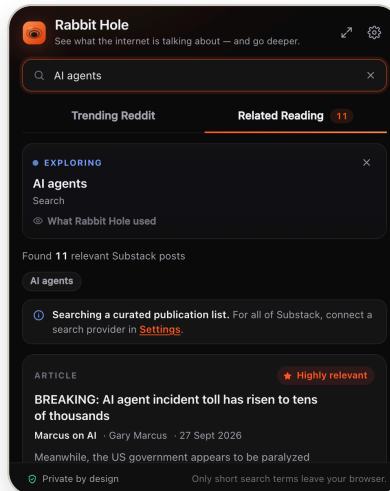


When Netflix hides its labels the extension says so and offers a manual box — it never guesses.

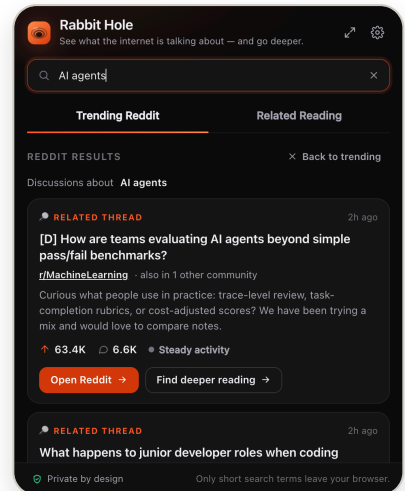
2.4 Search, loading and bridges



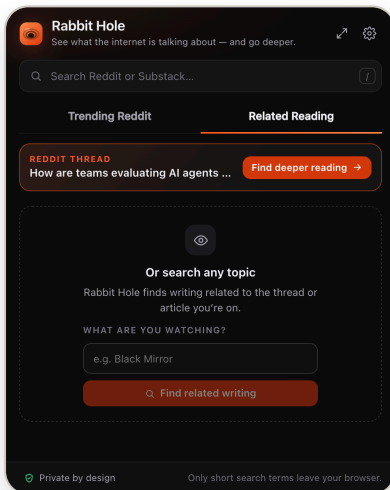
Loading: skeletons and “Finding related writing...” — never a blank screen.



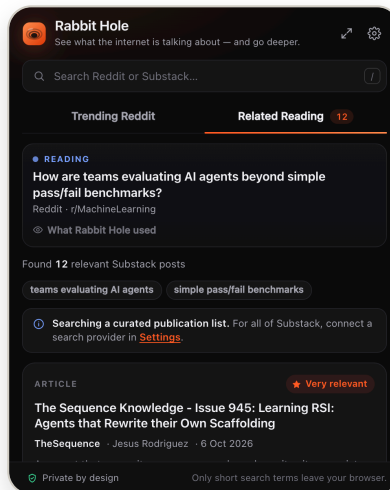
Results: “Exploring — AI agents”; N relevant posts.



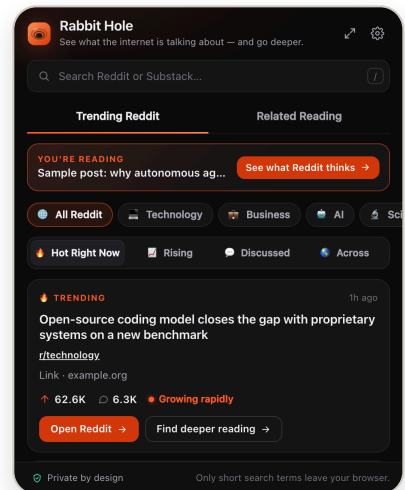
Same query on Reddit with “Back to trending”.



On a Reddit thread: “Find deeper reading →”.

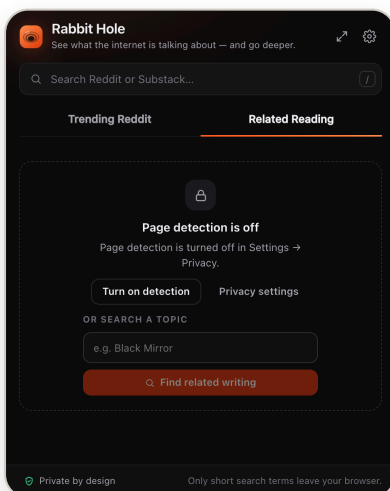


Result: thread headline becomes the search (“Reading”).

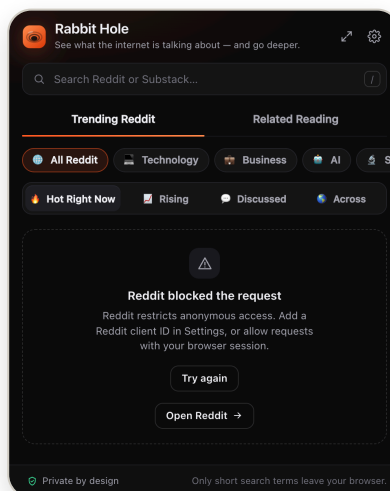


On a Substack article: “See what Reddit thinks →”.

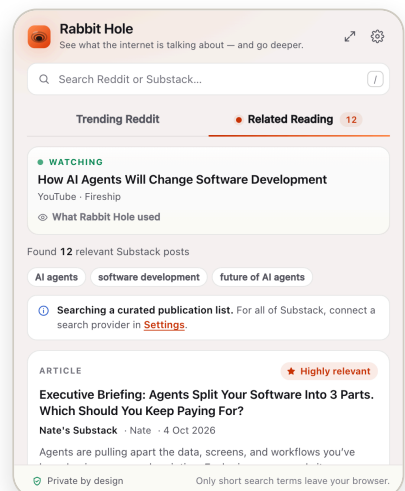
2.5 Privacy, errors and appearance



Detection off: the page is not read; offers to turn on or search.

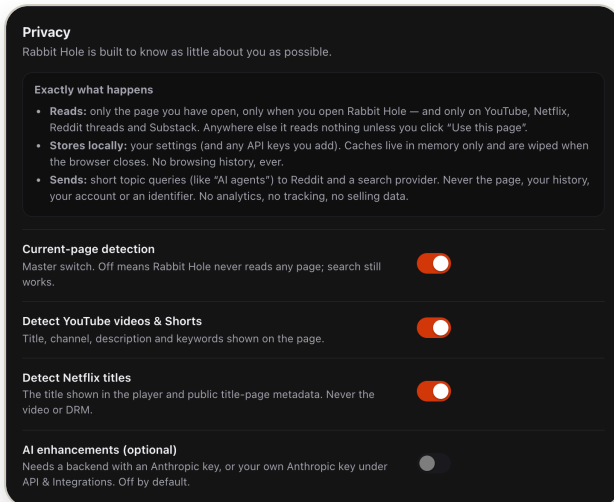


Error state: plain cause, one clear action.

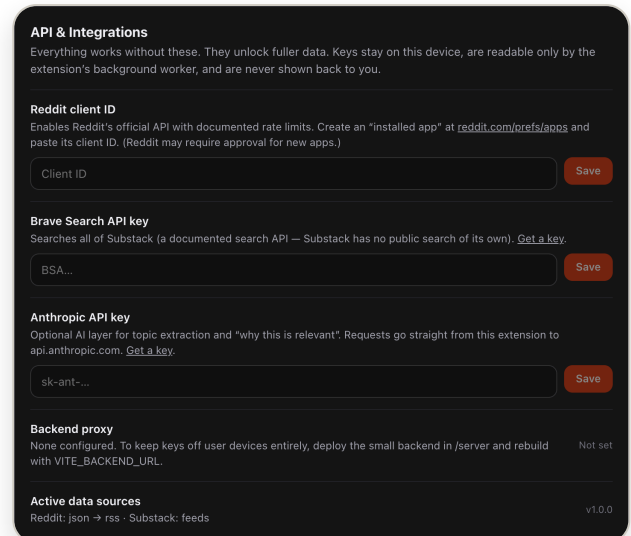


Light theme: dark by default; Light/System available.

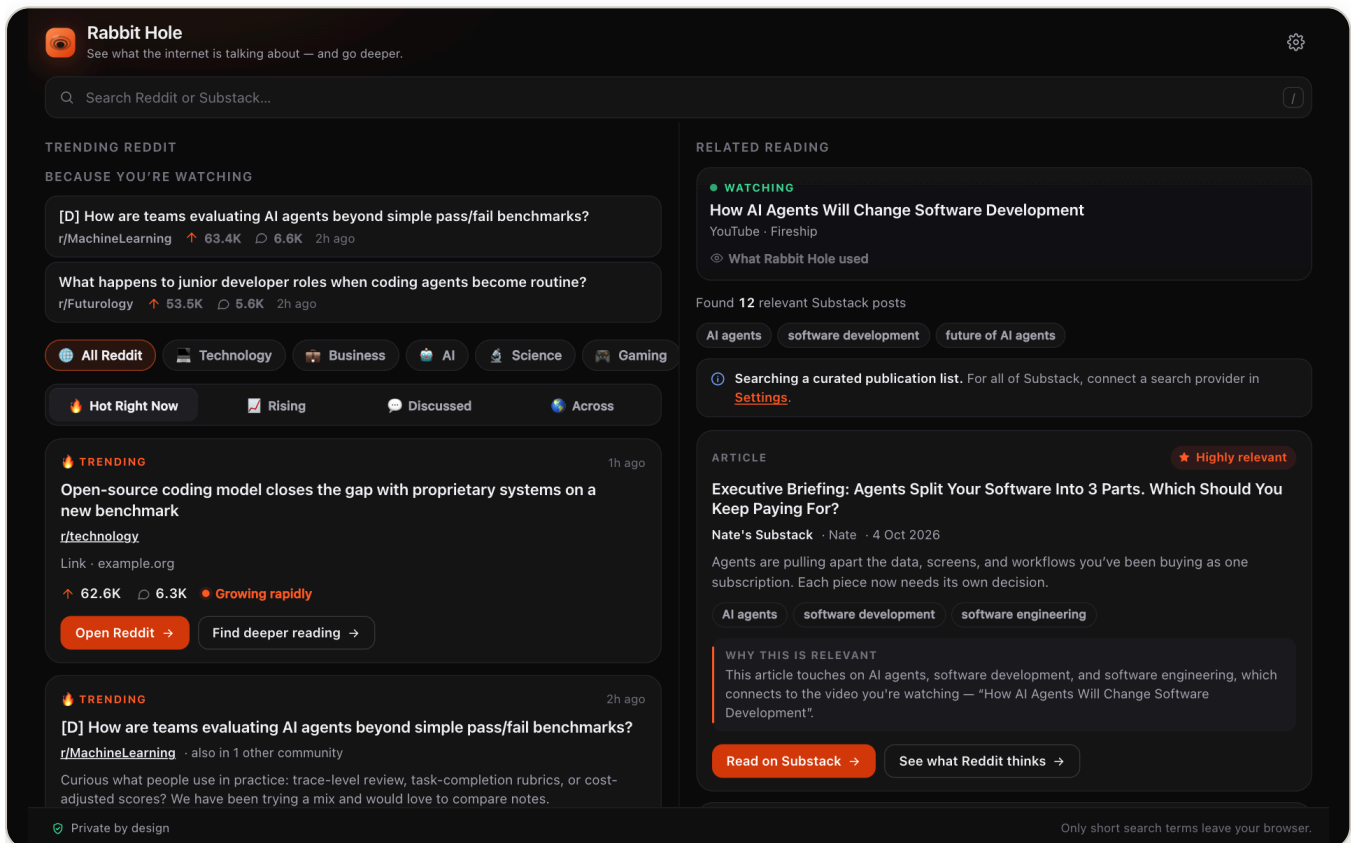
2.6 Settings and full dashboard



Settings → **Privacy** spells out what is read, stored and sent, with switches per platform and an opt-in for AI.



API & Integrations: keys are write-only; permissions requested only when a key is added.



Full dashboard (-): the same app in a tab, two columns when wide — Reddit left, Related Reading right. It exists because Chrome caps popups at 600 px tall.

UX and visual design

3.1 Design principles

- 1. **Understand, then act.** The first thing shown is *what Rabbit Hole understood* (the context bar), so users can trust or correct it.
- 2. **Never blank, never silent.** Skeleton loaders, a live status line and an explanation for every empty or failed state.
- 3. **Honest by default.** Missing data stays missing; limited coverage and feed-mode are announced; relevance is shown as a label, not a fake percentage.
- 4. **User in control.** Dismiss (x), Esc to clear, “Back to trending”, per-platform detection switches, delete-everything.
- 5. **Minimal chrome, premium feel.** Charcoal surfaces, one electric-orange accent, generous spacing, subtle motion (Arc / Linear / Readwise sensibility).

3.2 Visual language

Background
#0b0b0d
dark base

Surface
#141417
cards

Surface 2
#1a1a1f
insets

Text
#f4f4f6
primary

Text 2
#a8a8b3
secondary

Accent
#ff5a24
text / icons

Accent solid
#d63a0b
fills · 4.7:1 on white text

Good
#3fdc97
watching · success

Warn
#ffb633
rising

Cool
#7aa2ff
info · cooling

Aspect	Decision
Typography	System UI sans stack (SF / Segoe / Roboto) — no remote fonts (privacy, CSP, speed); 13.5 px base; tabular numerals for counts; weights 560–700 for hierarchy
Layout	480 × 600 popup: fixed header + search + tabs, scrolling panel, pinned privacy footer. Dashboard: max-width 1240 px, two columns ≥ 980 px
Shape & depth	10–18 px radii, 1 px hairline borders, no heavy shadows; hover raises border contrast
Motion	150–280 ms transitions; staggered-free “rise” on cards; pulsing live dot; all disabled under <code>prefers-reduced-motion</code>
Theming	CSS custom properties; dark default; Light and System; theme hint applied before first paint (no flash)
Iconography	Inline SVG line icons; emoji only for category labels (🔥 📈 💬 🌐) where they carry meaning

3.3 Accessibility measures built in

Area	Implementation
Keyboard	Every control is a real button/link; “/” focuses search; ←/→/Home/End move between tabs (roving tabindex); Enter submits; Esc clears
Screen readers	Tablist/tabpanel roles; <code>aria-live</code> status line (“Found N relevant posts”); labelled icon buttons; stats have text alternatives (“63400 upvotes”)
Focus	Visible 2 px accent focus ring on all interactive elements
Colour	Filled buttons use #d63a0b (4.7:1 with white); growth state is conveyed by text <i>and</i> a dot, never colour alone
Motion	Animations disabled for reduced-motion users
Target size	Buttons 28–32 px tall; icon buttons 32 × 32 (WCAG 2.2 minimum 24 px)

A formal WCAG 2.2 AA audit is part of the evaluation pack (Method F); the above are design intentions, not a certification.

System design

4.1 Architecture at a glance

The extension has three runtime contexts. **Content scripts** are tiny and passive: they do nothing until asked, then read the page once and reply. **Extension pages** (popup, dashboard, settings) are a React app that never touches the network. The **background service worker** owns everything else — all network access, caching, key handling, ranking — and talks to the pages over a typed, cancellable RPC port. The full diagram is on the next page.

4.2 The context pipeline

Stage	Module	What it does
1 · Platform detector	<code>context/platform.ts</code>	Pure URL logic: YouTube watch/Shorts/live, Netflix watch/title, Reddit thread, Substack article; look-alike hosts rejected
2 · Metadata extractor	<code>context/extractors/*</code>	DOM → <code>RawPageMetadata</code> in the page: visible title/channel/description/keywords (YouTube), player label + JSON-LD (Netflix), thread markup (Reddit)
3 · Topic extraction	<code>context/topics.ts</code>	Clean title → RAKE-style phrases (stop-words and filler verbs act as delimiters) → weighted by source (title 3, keywords ~1.9, description 1)
4 · Entities	<code>context/topics.ts</code>	Brands/people/titles; Title-Case aware so “AI Agents” is a topic, not a name; known-entity list; cast from JSON-LD
5 · Concept expansion	<code>context/ontology.ts</code>	On-device concept graph (“AI agents” → AI coding, developer tools, automation ...); genre → themes; or the AI layer
6 · Query generation	<code>context/queries.ts</code>	3–5 focused, de-duplicated queries (entity-anchored, topic-anchored, concept-anchored); AI queries first when enabled
7 · Discovery	<code>substackService.ts</code>	Search providers (backend / Brave key) in parallel; curated RSS feeds as keyless fallback and supplement
8 · Ranking	<code>ranking/relevance.ts</code>	Transparent 0–100 score, content-gated labels, diversity re-rank
9 · Results	React UI	Cards with “Why this is relevant”

4.3 Ranking algorithms

Reddit trending score

$$\text{trending} = \text{recency} \times \text{engagementVelocity} \times \text{commentVelocity} \times \text{popularity} \times \text{growth}$$

- **recency** = $0.25 + 0.5^{(\text{ageHours} / 10)}$ — fresh threads dominate, old ones fade
- **engagementVelocity** = $1 + \log_{10}(1 + \text{score} / (\text{age} + 1))$ · **commentVelocity** = $1 + \log_{10}(1 + \text{comments} / (\text{age} + 1))$
- **popularity** = $1 + 0.12 \cdot \log_{10}(1 + \text{score}) + 0.12 \cdot \min(\text{cross-community}, 5) + 0.08 \cdot (\text{extra listings}) + 0.15 \cdot (\text{preferred subreddit})$
- **growth** = $1 + 0.33 \cdot \log_{10}(1 + \text{measured upvotes/hour} \div 10)$, measured against the previous refresh snapshot

Article relevance (0–100)

$$\text{relevance} = 100 \times (0.45 \cdot \text{semantic} + 0.20 \cdot \text{keyword} + 0.15 \cdot \text{entity} + 0.10 \cdot \text{recency} + 0.10 \cdot \text{sourceQuality})$$

- **semantic**: concept-weighted cosine similarity over stemmed terms, saturated to 0–1; common stems (“ai”, “data”) down-weighted like a cheap IDF
- **keyword / entity**: phrase and name matches, title matches weighted higher; entity weight is redistributed when the content has no named entities
- **recency**: $e^{-\text{age}/150 \text{ days}}$, unknown dates get a neutral value — never a guessed date
- **Labels**: Highly relevant ≥ 70 · Very relevant ≥ 52 · Related ≥ 36 · else Somewhat related — and the top two also require genuine content overlap, so freshness and source quality alone can never earn “Highly relevant”

4.4 Data sources and graceful degradation

Source	Providers (tried in order)	When a provider fails
Reddit	① Official OAuth API (client id) · ② public JSON · ③ official Atom feeds	Circuit breaker skips a blocked/rate-limited provider for minutes; feeds carry no counts, so the UI says so and ranks by Reddit’s own order
Substack search	① Optional backend proxy · ② Brave Search API key (<code>site:substack.com</code>)	Falls back to curated RSS; reports which providers failed
Substack feeds	61 curated publications (33 custom-domain origins via opt-in permission); newest 12 items, streamed	Per-feed failures ignored; error only if all fail
AI (optional)	Backend or user’s Anthropic key; structured JSON, low effort, refusal fallbacks	Silently degrades to on-device heuristics

4.5 Caching and performance

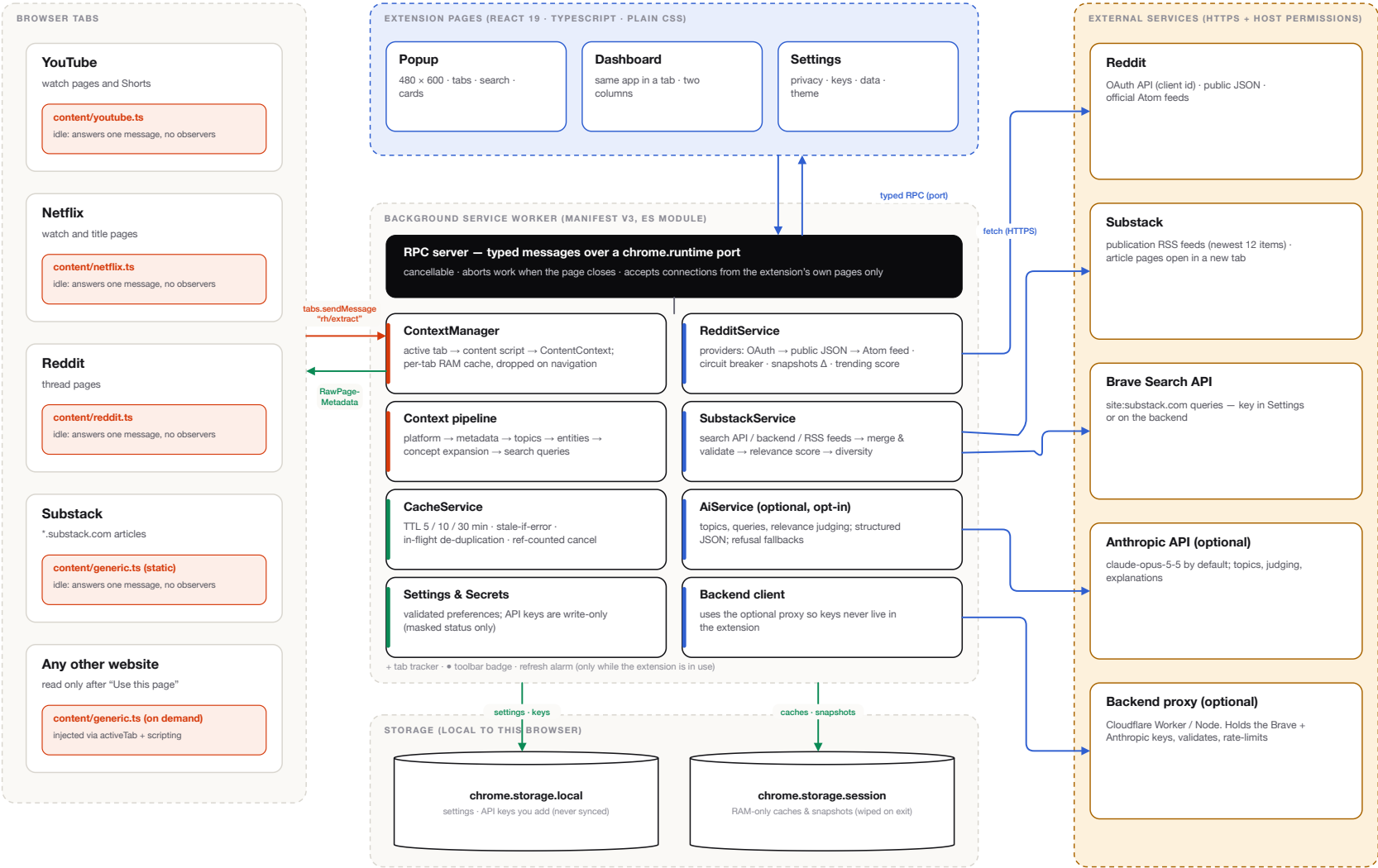
Data	TTL	Behaviour
Reddit trending / search	5 min	Cache-first paint; refresh in background; stale-if-error (up to 6 h, flagged)
Substack search	10 min	Keyed by context + settings
Publication feeds	30 min	Only the first 12 items are downloaded (feeds can be 2 MB+)
AI analysis	60 min	Keyed by the exact (truncated) input
Page context	Until the tab navigates	Per-tab, RAM only (<code>storage.session</code>)
In-flight requests	—	Identical concurrent calls share one fetch; cancellation is reference-counted

Other performance measures: debounced search (450 ms), cancel-stale-request on every new search, skeleton paint before data, no polling (the only timer is an optional background refresh, paused after two idle hours), lazy per-tab fetching, 6–9 KB content scripts.

4.6 System architecture diagram

Rabbit Hole — system architecture

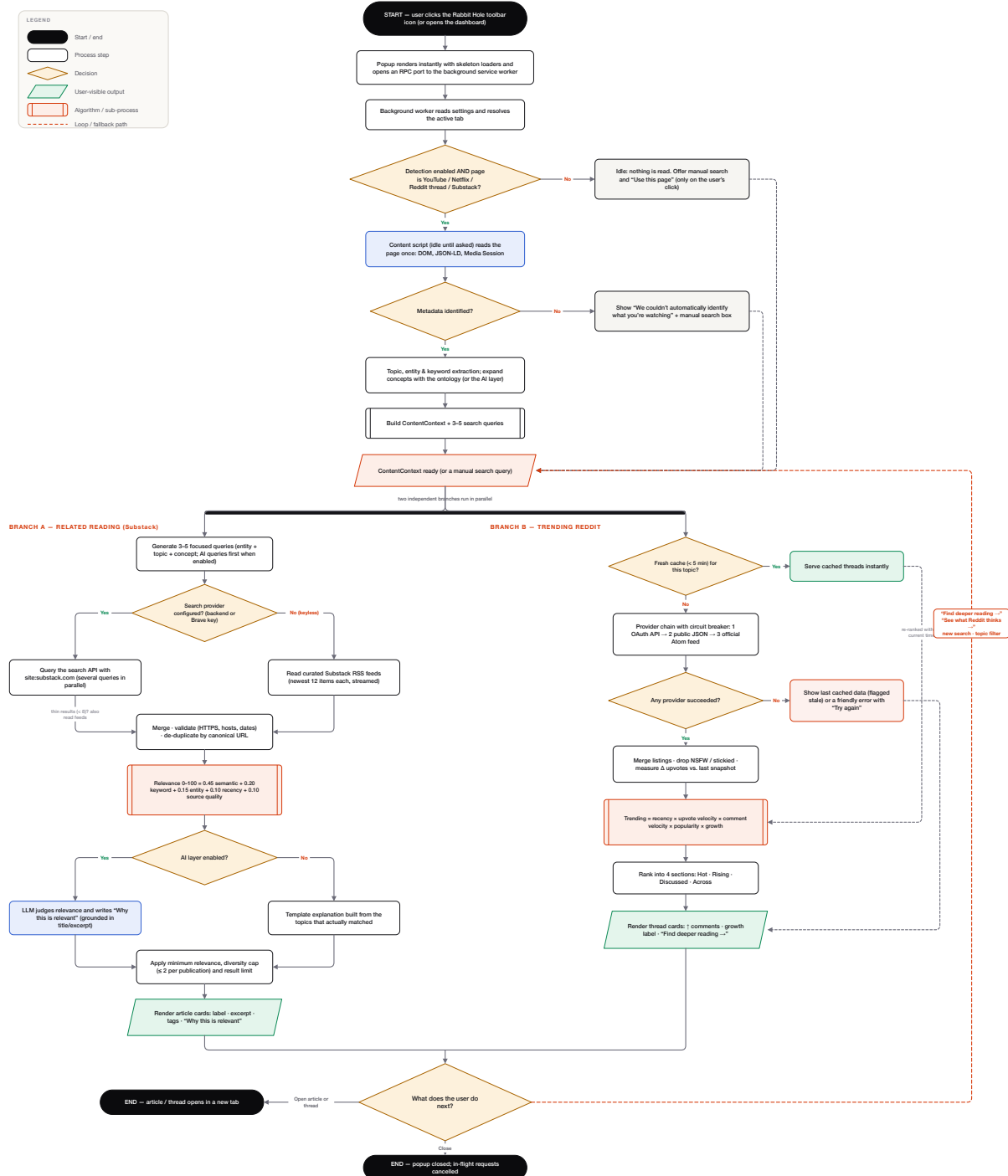
Manifest V3 · all network access lives in the background worker · pages and content scripts never talk to the internet



Flowcharts

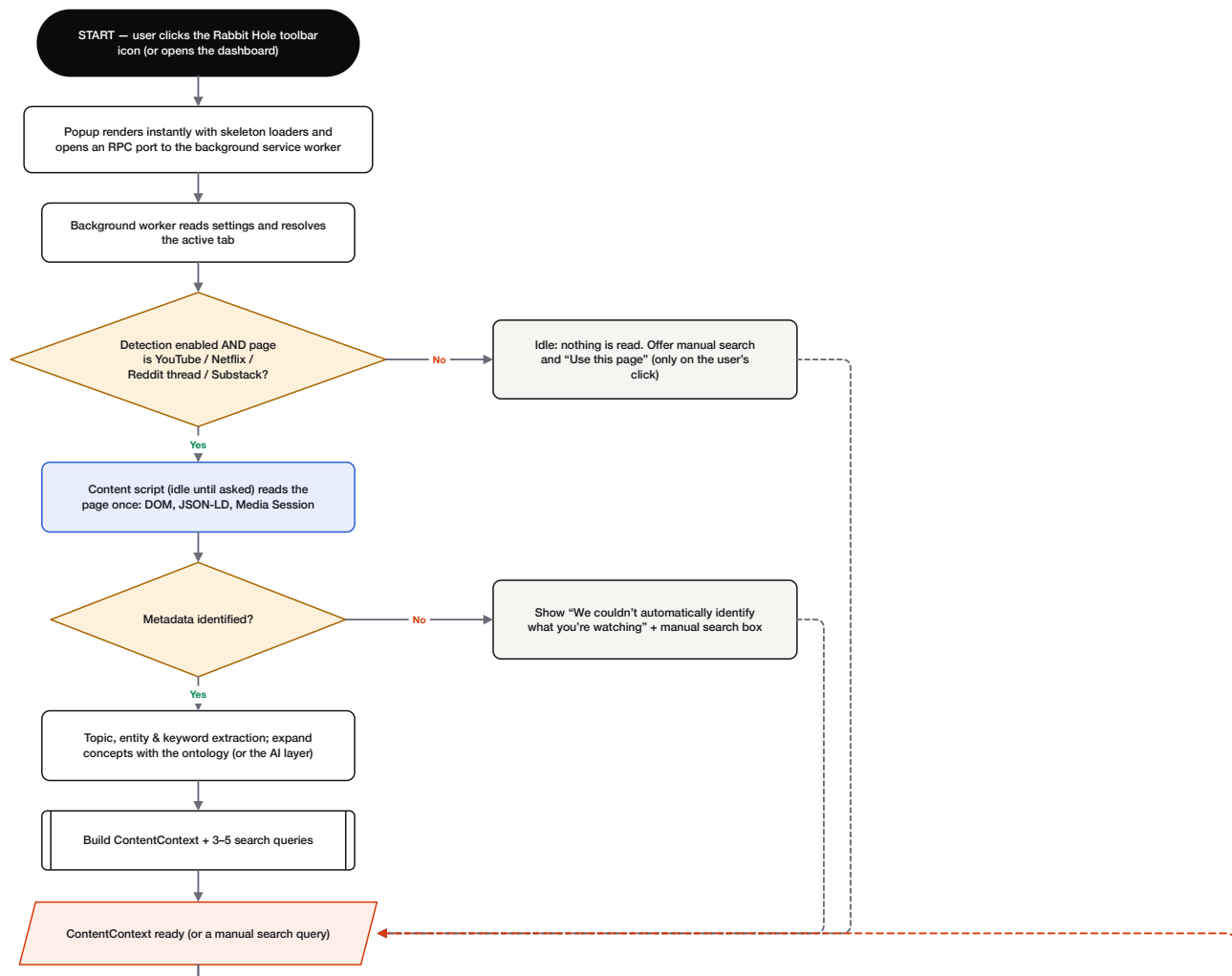
Three diagrams document the project; each exists as a vector SVG in `docs/flowcharts/`. The project flowchart (this section) follows standard flowchart symbols; the architecture diagram is in §4; the UI flow — built from the real screens and ready to import into Figma — closes this section.

5.1 Project flowchart — overview



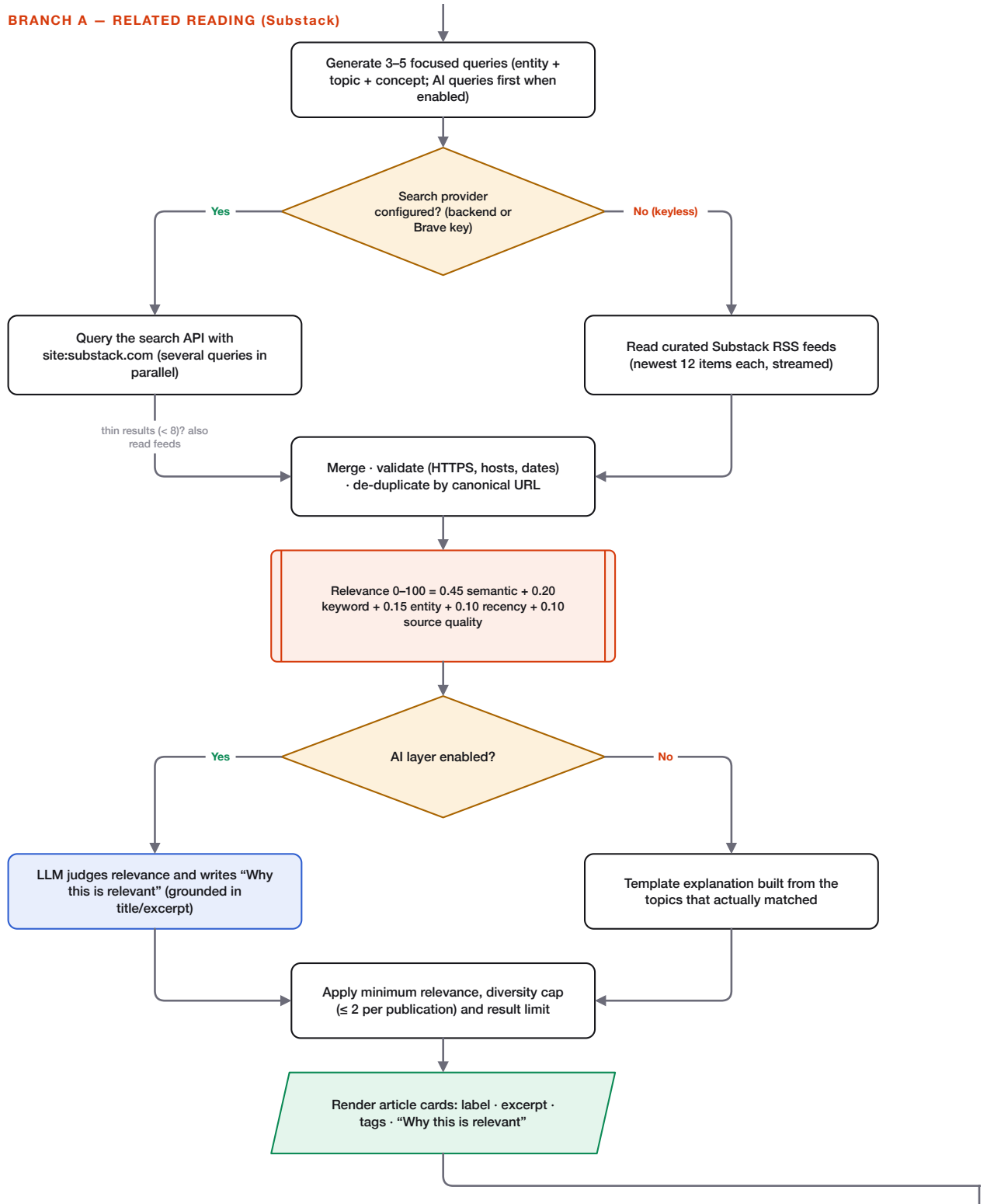
The next three pages zoom into each part at a readable size.

5.2 Part 1 — Capturing the context



The popup paints skeletons immediately while the background resolves the active tab. A page is read **only** if detection is on *and* the platform is supported; otherwise the user gets manual search and an explicit “Use this page” button. Both fallbacks rejoin the pipeline at the same `ContentContext` node, so downstream code never cares how the context was obtained.

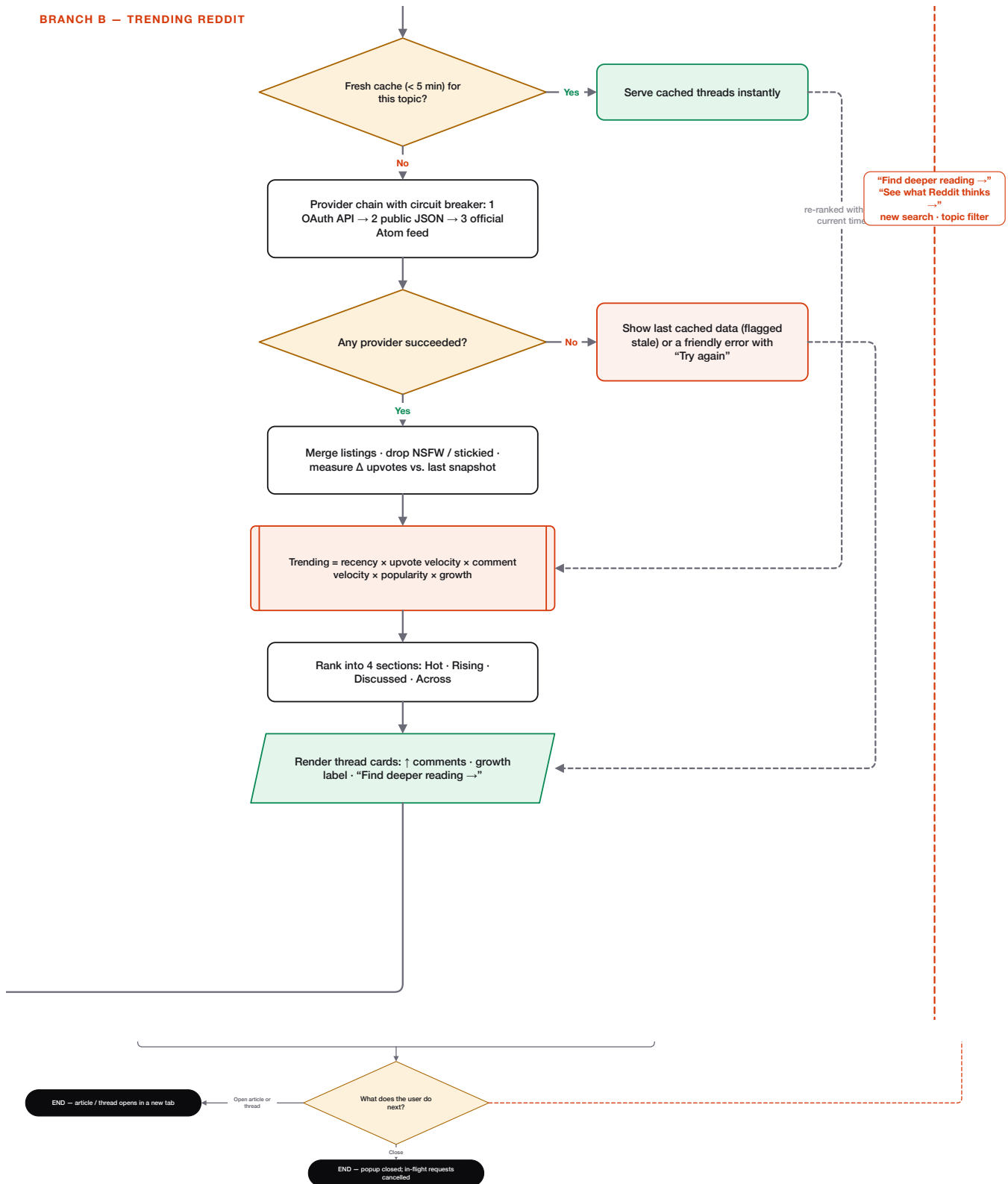
5.3 Part 2 — Branch A: Related Reading



With a search key or backend, queries run against a search API; without one, curated RSS feeds are read (and also used to supplement thin results). Everything is merged, validated and scored; the optional AI layer judges the front-runners and writes the explanation, otherwise a template names the topics that actually matched.

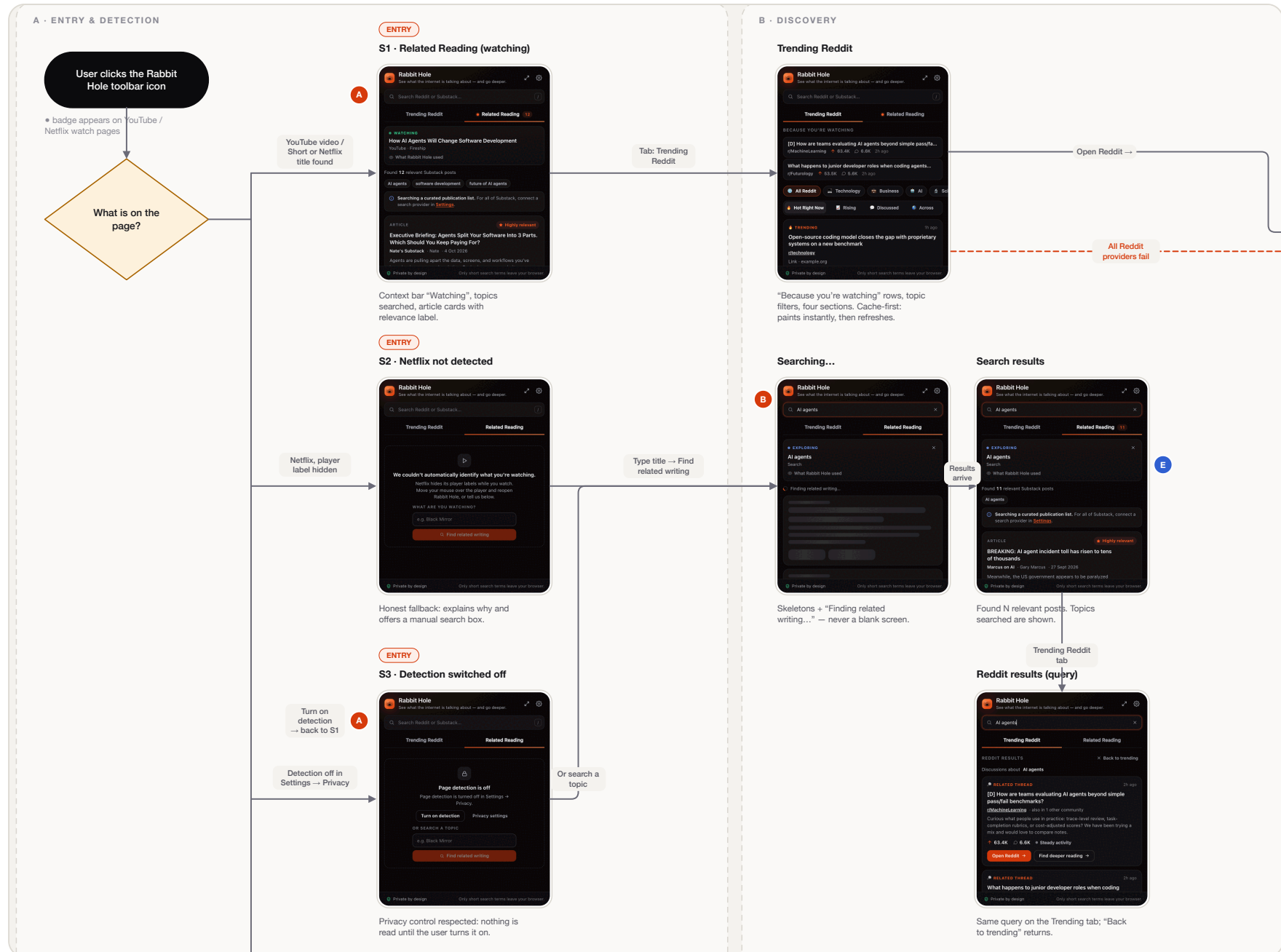
5.4 Part 3 — Branch B: Trending Reddit, and what the user does next

BRANCH B — TRENDING REDDIT

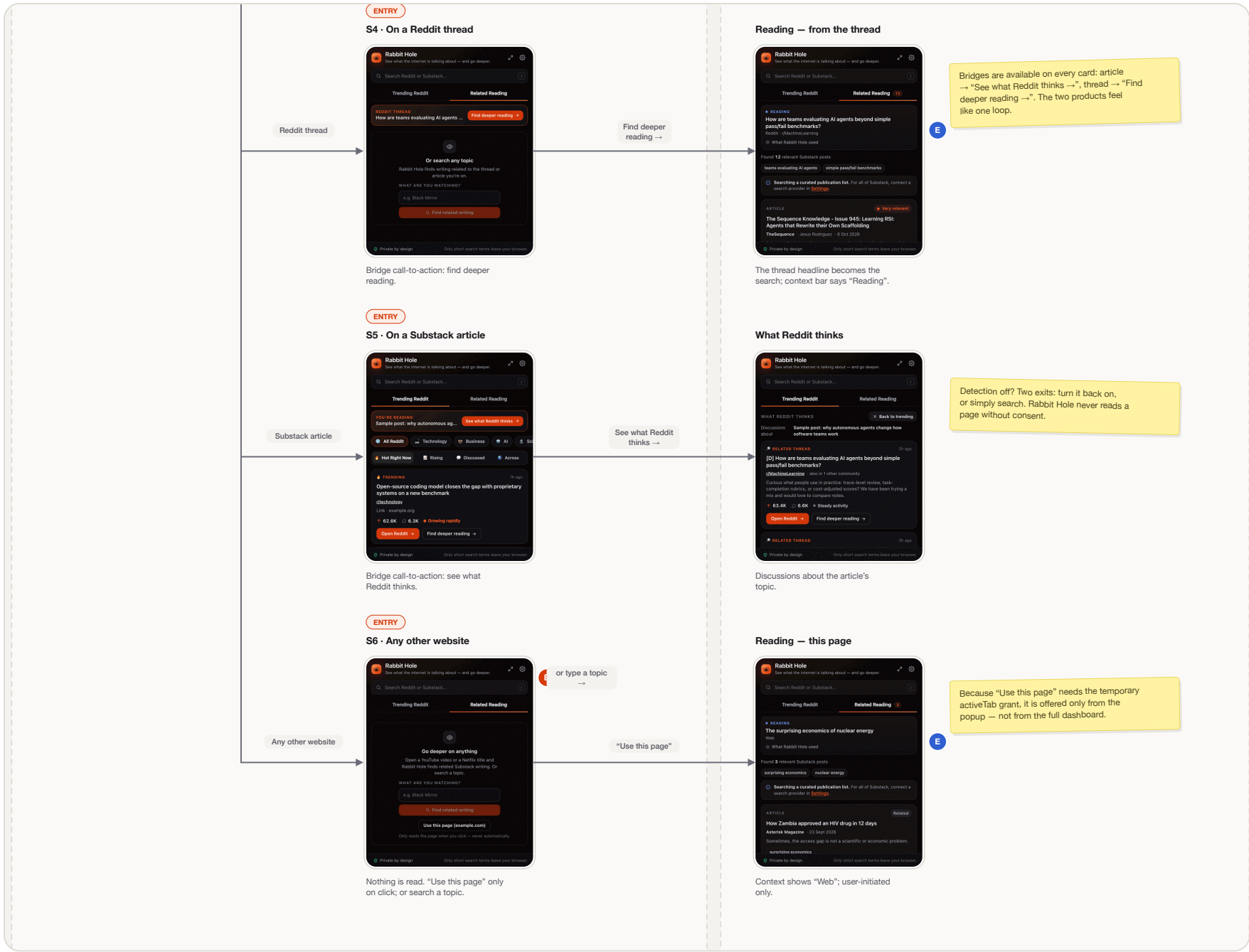


Cached data renders instantly and is re-ranked with the current time. On a miss the provider chain runs behind a circuit breaker; if every provider fails, the last good data is shown (flagged stale) or a friendly error with a retry. The dashed loop-back shows the bridge buttons and new searches starting a fresh context.

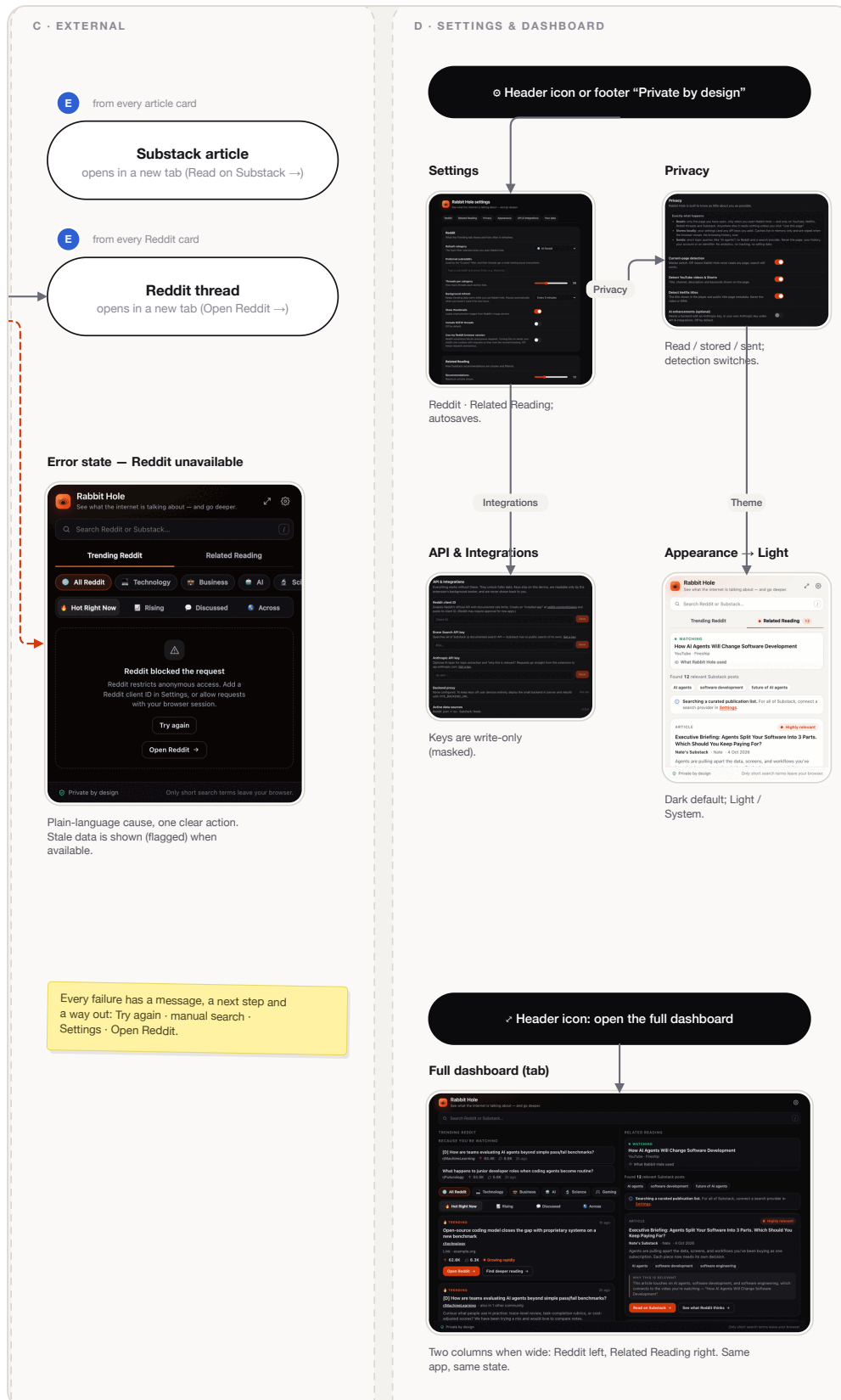
5.5 UI flow (Figma-ready) — A · entry & detection, B · discovery · rows S1–S3 · file: docs/flowcharts/figma-ui-flow.svg



5.5 UI flow — rows S4–S6: bridges between Reddit and Substack, and “Use this page” · always user-initiated



5.5 UI flow — C · external results, D · settings & dashboard



Using it in Figma: drag `figma-ui-flow.svg` onto a Figma or FigJam canvas (or File → Import). Frames, labels and arrows arrive as separate layers and the screenshots as image fills. `ui-flow.mmd` is a Mermaid version for native FigJam diagrams.

Privacy and security

6.1 Privacy by design

Question	Answer
What is read?	Only the page the user has open, only when the extension is opened, and only on YouTube, Netflix, Reddit threads and Substack. On any other site nothing is read unless the user clicks “Use this page” (using the temporary <code>activeTab</code> grant).
What is stored?	Settings and any API keys the user adds (<code>storage.local</code> , never synced). Caches and per-tab context live in <code>storage.session</code> — RAM only, gone when the browser closes. No browsing history is kept.
What is sent?	Short topic queries (“AI agents”) to Reddit and a search provider. With the AI layer on (off by default): a title, channel and ≤ 500 characters of description. Never page contents, URLs, history or identifiers.
Tracking / analytics?	None. No third-party scripts, fonts or telemetry. Nothing is sold.
User control	Master and per-platform detection switches; “What Rabbit Hole used” disclosure on the context bar; clear cache; delete everything; revoke optional permissions.

6.2 Security controls

Threat	Control	Verified by
Malicious page text (XSS)	No HTML sinks: all external text is rendered as React text nodes after control/bidi-character stripping and length caps; HTML is converted to inert text	Source scan; parser/sanitise tests
Script injection / remote code	CSP <code>script-src 'self'; object-src 'self'; base-uri 'none'</code> ; no eval, inline or remote scripts	Release gate scans build; real-Chrome run: 0 CSP violations
Malicious or compromised data source	Every response validated; links must be HTTPS on allow-listed hosts; look-alike hosts rejected; implausible dates dropped	parsers, sanitise, backend tests
Key theft	Keys write-only from the UI (masked status), stored apart from settings, <code>storage.local</code> locked to trusted contexts, never in URLs; dev keys stripped from production builds	Release gate with sentinel keys; secrets tests
Hostile web page talking to the extension	Background accepts RPC only from the extension’s own pages; content scripts reply only to the extension and only to one message type	rpc tests (foreign sender refused)
Prompt injection (AI layer)	Page metadata fenced and declared untrusted; schema-constrained output re-validated; explanations must be grounded in the supplied excerpt	anthropicProvider tests
Over-broad permissions	Four permissions; hosts limited to five sites; APIs and 33 custom-domain feeds only as <i>optional</i> , per-feature, user-granted permissions	manifest tests; real-Chrome check
Backend abuse	Origin allow-list, per-client rate limit, size caps, no key echo; honest note that origin checks are not authentication	backend tests

Testing and quality assurance

7.1 Automated tests — 230 passing in 17 files

Area	Tests	What is covered
Settings, secrets, sanitising	20	clamping, write-only keys, URL allow-lists, HTML → inert text, stemming, formatting
API failure handling (HTTP)	19	401/403/404/429/5xx mapping, Retry-After, offline, timeout, cancel, invalid JSON, size limits, partial feed reads
Topic extraction & query generation	19	spec examples, Title-Case entities, tag stripping, Netflix themes, manual searches, AI-first queries
API failure handling (services)	18	Reddit OAuth → JSON → RSS chain, circuit breaker, stale data, Substack degradation, empty results
Relevance scoring	18	formula weights, 0–100 bounds, labels, content gating, explanations, diversity, AI blending
Cache behaviour	17	TTLs, stale-if-error, in-flight de-duplication, ref-counted cancel, eviction, quota failure
Parsing & validation	17	Reddit JSON/Atom, RSS, search results, URL/date/author sanitising, merge & de-duplicate
Platform detection	15	YouTube/Netflix/Reddit/Substack URL rules, look-alike hosts, junk input
Backend proxy	14	CORS, rate-limit, validation, no key leakage, extension ⇔ backend contract
Reddit ranking	13	trending score, growth classification, cross-community counts, four categories, feed-only mode
AI provider	12	request shape, prompt-injection fencing, structured output, refusals, error mapping
Netflix metadata extraction	12	player label, movie vs episode, JSON-LD, Media Session, title-page enrichment, hidden-controls fallback
Manifest & build artefact	11	minimal permissions, no broad hosts, strict CSP, referenced files exist
Context manager	10	consent rules, per-tab cache, privacy switches, undetected/idle states, badge logic
RPC security	8	typed round-trips, cancellation, worker loss, prototype-method tricks, foreign senders refused
YouTube metadata extraction	6	watch page, Shorts, stale SPA, tab-title fallback, registry safety
Built service worker smoke	1	boots dist/background.js with no DOM and serves RPC

7.2 Release gate and real-browser end-to-end

Release gate (`npm run verify:release`): builds with sentinel secrets in the environment and proves they cannot appear in a production bundle; confirms the public client id and backend origin *are* compiled in; scans

for `eval`, `new Function`, `document.write`, inline/remote scripts and source maps; checks content scripts are self-contained.

Real-Chrome end-to-end (`npm run test:e2e`, Chrome for Testing 155): loads the built `dist/`, serves fixture pages at the *real*/YouTube/Netflix/Reddit URLs so the real content scripts inject, and drives the real popup and Settings against the real service worker — **28 of 28 checks pass** (stable over repeated runs). Highlights:

- Service worker boots from the built manifest; permissions are exactly the minimal four
- Extraction on a video, a Short, a Netflix episode and a Reddit thread; hidden Netflix labels degrade to “unknown”
- • badge appears on watch pages and clears when detection is turned off
- Live Substack feeds fetched from the service worker through host permissions (12 real posts)
- The extension **cannot see the URL** of an ordinary website’s tab and no content script runs there
- Settings toggles persist; with detection off the page is not read; secrets RPC returns masked status only
- Zero Content-Security-Policy violations; zero uncaught errors

7.3 Defects found and fixed during development

Reviewing real captures of the UI and running the real-browser suite surfaced genuine defects — each now has a fix and, where practical, a regression test.

#	Finding	Found by	Resolution
1	Publication topic tags leaked into article scoring, handing every post from an “AI” newsletter a free match	Ranking review of real results	Tags now only select feeds; test asserts no effect on score
2	“Highly relevant” reachable from source quality + freshness alone	Same review	Top labels require content overlap (gated)
3	Generic stem “ai” dominated matching; roundups outranked on-topic posts	Ranking review	Ubiquitous stems down-weighted (cheap IDF)
4	Reddit JSON blocked (403) and feeds rate-limited (429) from the build network	Live probing	Three-provider chain, circuit breaker, stale-if-error, honest feed mode
5	Two “x” buttons in the search box (native + custom)	Screenshot review	Native cancel button hidden
6	Reddit tag “[D] ...” shown as “D ...” in the context bar	Screenshot review	Leading post tags stripped; test added
7	Toolbar • badge stayed on open tabs after detection was switched off	Real-Chrome e2e	Badge recomputed for every open tab on settings change
8	<code>agents.</code> became “agents .” (inline tags added spaces)	Parser tests	Inline tags removed without a space
9	“code” and “coding” stemmed differently, hurting matching	Text tests	Stemmer fixed; test added
10	Popup opened on Trending, then flipped to Reading ~100 ms later on video pages	Screenshot timing	First paint held on a skeleton until the tab is decided
11	Substack feeds up to 2 MB each downloaded in full	Network inspection	Streamed read stops after 12 items
12	“AI Agents” in a Title-Case headline detected as a named entity	Topic probe on the spec’s example	Title-Case-aware entity rules; known-entity list

Usability evaluation plan

The product has been verified technically; whether people *understand and enjoy* it needs usability evidence. The accompanying **Heuristic & Usability Evaluation Form** (a fillable PDF, [docs/Rabbit-Hole-Heuristic-Evaluation-Form.pdf](#)) operationalises the plan below.

Method	Participants	Output	Primary measure
A · Heuristic evaluation (Nielsen's 10)	3–5 evaluators (UX / dev / peers)	Severity-rated problems per heuristic	Mean severity 0–4; # major problems
B · Cognitive walkthrough	2–3 evaluators	Step-level learnability failures	4 questions × steps for 7 tasks
C · Moderated think-aloud test	5 target users	Observer log per task	Completion, time, errors, SEQ (1–7)
D · SUS questionnaire	Same 5 users	Standardised usability score	SUS 0–100 (average ≈ 68)
E · Relevance & trust survey	Same 5 users	Precision of recommendations; trust/understanding of privacy	Precision@5; Likert 1–5; NPS
F · Accessibility audit (WCAG 2.2 AA)	1–2 evaluators + tools	Pass/fail checklist	# failed criteria
G · Error-state & performance matrix	1–2 evaluators	Expected vs observed behaviour	Pass rate; time to first paint / results
H · A/B experiment plan	≥ 20 users per variant (future)	Decisions on four open design questions	Click-through; time to first click

8.1 Design hypotheses to test

The design team enters the evaluation with explicit, falsifiable suspicions (listed in the form's facilitator notes and withheld from evaluators until they have finished independently, to avoid bias):

H-1 New users will not know what “Highly relevant” is based on; the tooltip that explains it is hidden.

H-2 There is no first-run explanation of what the extension reads; privacy clarity depends on discovering the footer and Settings.

H-3 Cards are dense at 480 × 600; the “Why this is relevant” box may be skipped.

H-4 Netflix users whose labels are hidden may not notice that the fallback box is the way forward.

H-5 Emoji-based section names (🔥 📄 💬 🌐) may be ambiguous or noisy for screen-reader users.

H-6 The automatic tab choice (Related Reading when a video is detected) may surprise users who expect Reddit first.

H-7 “Because you’re watching” rows can show weak matches, harming trust in the whole tab.

H-8 The Settings page (six sections) is long; users may not find “Delete everything” or the key fields.

8.2 Success criteria (proposed)

Metric	Target
Task completion (core tasks T1–T5)	≥ 80 % unassisted
T1 time (open on a video → open a related article)	≤ 30 s median
SUS	≥ 75 (“good”)
Precision@5 of recommended articles (judged “relevant” or “partly”)	≥ 70 %
Heuristic evaluation: major (3) or catastrophic (4) problems	0 unresolved before release
Accessibility audit: failed WCAG 2.2 AA criteria	0 critical
“I understand what Rabbit Hole reads and sends” (Likert 1–5)	≥ 4.0 mean

Limitations, risks and future work

9.1 Known limitations (stated plainly)

- **Popup height is capped by Chrome at 600 px** (the brief suggested 650–750); the full dashboard tab provides the roomy layout.
- **Reddit access is outside our control.** Anonymous JSON may be blocked; without a client id the extension falls back to Atom feeds that carry *no* counts. Live Reddit data could not be verified from the build machine.
- **Substack has no public search API.** Without a search key or backend, coverage is a curated list of 61 publications; posts on custom domains cannot be found through `site:substack.com`.
- **Netflix markup changes without notice** and hides its labels during playback. Detection is best-effort and falls back to manual search; selectors were verified against fixtures, not a live subscription.
- **Semantic relevance is lexical** without the AI layer; it cannot know two differently-worded ideas are the same. The AI layer is unit-tested against a stubbed transport only (no key was available for a live call).
- **Usability is untested with users.** The evaluation form exists precisely to close that gap.

9.2 Future work

Priority	Item	Why
High	First-run onboarding (what is read / sent) and a visible explanation of relevance labels	Addresses H-1, H-2
High	Run the evaluation pack with real users; iterate on severity ≥ 3 findings	Closes the usability evidence gap
High	Reddit client-id onboarding flow and Chrome Web Store packaging / privacy policy page	Reliable data + distribution
Medium	“Not relevant” feedback stored locally to personalise ranking	Better precision without tracking
Medium	Embedding-based semantic scoring behind the backend	Replace lexical approximation
Medium	More platforms (podcasts, Spotify, Twitch) via the extractor interface	One extractor + one URL rule each
Low	Firefox / Safari builds (WebExtensions), localisation, notifications for followed topics	Reach

Appendices

A · Permissions, explained

Permission	Why
<code>storage</code>	Settings (local), API keys you add (local, background-only) and RAM-only caches (session).
<code>activeTab</code>	Lets “Use this page” read one tab on any site, only after a click.
<code>scripting</code>	Injects the small reader on demand (tabs opened before install; “Use this page”).
<code>alarms</code>	Optional background refresh of trending data, paused after two idle hours.
<code>https://www.reddit.com/*</code>	Reddit data (feeds / JSON / official API)
<code>https://oauth.reddit.com/*</code>	Reddit data (feeds / JSON / official API)
<code>https://www.youtube.com/*</code>	Content script: “what is playing?”
<code>https://m.youtube.com/*</code>	Content script: “what is playing?”
<code>https://www.netflix.com/*</code>	Content script: “what is playing?”
<code>https://*.substack.com/*</code>	Publication RSS feeds; content script on Substack pages
<code>optional: Brave, Anthropic</code>	Requested at runtime only when the user adds that key
<code>optional: 33 custom-domain feeds</code>	Requested at runtime only when “Include custom-domain newsletters” is turned on

Not requested: `tabs`, `history`, `cookies`, `webRequest`, `<all_urls>`.

B · Technology stack

Layer	Technology	Version
Platform	Chrome Extension, Manifest V3 (ES-module service worker)	Chrome ≥ 116
Language	TypeScript (strict, noUncheckedIndexedAccess)	^7.0.2
UI	React · plain CSS design tokens (no CSS framework)	^19.3.0
Build	Vite (app) + per-script IIFE builds for content scripts	^8.3.3
Tests	Vitest · jsdom · puppeteer-core (e2e)	^5.0.3 · ^30.1.2 · ^25.12.0
Parsing	fast-xml-parser (RSS / Atom)	^5.11.2
AI (optional)	@anthropic-ai/sdk — structured JSON output	^0.131.0
Backend (optional)	Web-standard handler on Cloudflare Workers / Node	—

C · Repository layout

```
manifest.config.mjs  MV3 manifest generator
src/
```

background/	service worker: container, RPC handlers, contextManager, tabs
content/	youtube.ts · netflix.ts · reddit.ts · generic.ts
popup/ options/	React app + Settings page
components/	cards, tabs, search, skeletons, states
services/	reddit · substack · context · ranking · cache · ai · settings · secrets
utils/ types/	http, rpc, sanitize, text, errors · shared contracts
server/	optional backend proxy
tests/	17 test files (+ e2e/)
docs/	this report, evaluation form, flowcharts, screens

D • Glossary

Term	Meaning
Context	The normalised description of what the user is looking at (<code>ContentContext</code>)
Provider	One interchangeable data source (e.g. Reddit OAuth, Reddit JSON, Reddit Atom)
Circuit breaker	Temporarily skipping a provider that just failed so it is not hammered
Stale-if-error	Serving expired cached data (flagged) when a refresh fails
Heuristic evaluation	Expert inspection of a UI against usability principles
SUS	System Usability Scale — 10-item questionnaire scored 0–100
SEQ	Single Ease Question — 1–7 rating after each task

E • References

1. Nielsen, J. (1994). *10 Usability Heuristics for User Interface Design*. Nielsen Norman Group. nngroup.com/articles/ten-usability-heuristics
2. Nielsen, J., & Molich, R. (1990). Heuristic evaluation of user interfaces. *Proc. ACM CHI '90*, 249–256.
3. Nielsen, J. (1994). Severity ratings for usability problems. Nielsen Norman Group.
4. Lewis, C., & Wharton, C. (1997). Cognitive walkthroughs. In *Handbook of Human-Computer Interaction* (2nd ed.). Elsevier.
5. Brooke, J. (1996). SUS: A “quick and dirty” usability scale. In *Usability Evaluation in Industry*. Taylor & Francis.
6. Sauro, J., & Lewis, J. R. (2016). *Quantifying the User Experience* (2nd ed.). Morgan Kaufmann.
7. W3C (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. w3.org/TR/WCAG22
8. Google. *Chrome Extensions — Manifest V3*. developer.chrome.com/docs/extensions
9. Reddit. *Data API documentation*. reddit.com/dev/api · Brave. *Search API*. brave.com/search/api
10. Anthropic. *Claude API documentation*. docs.anthropic.com